



Pashov Audit Group

Paxos Labs Security Review

June 25th 2026 - June 27th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. About Paxos Labs	4
6. Executive Summary	4
7. Findings	5
Low findings	6
[L-01] Stale <code>executePendingOrders</code> signature prevents merkle verified execution	6
[L-02] The protocol may charge zero fees for low-decimal stable coins	6



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. Methodology

Pashov Audit Group conducts each engagement as a team that works through the codebase end-to-end: we map the system, hunt for bugs, and then collaborate with the client to remediate the findings. This report tracks every commit reviewed, every finding raised, and the resolution status of each issue. Our auditors draw on experience from private engagements and public contests, and pick their own toolset per project — static analysis, fuzzers, formal verification, and AI-assisted review — based on what fits the codebase.

5. About Paxos Labs

Paxos Labs is a vault management platform built on the BoringVault architecture that enables cross-chain asset operations and multi-asset deposits. This review covers the Transit Station order execution system with LayerZero cross-chain bridging, the Equivalent Exchange module for quote-based swaps, and new decoders/sanitizers for Uniswap Universal Stablecoin and Khalani.

6. Executive Summary

A time-boxed security review of the **paxoslabs/nucleus-boring-vault** repository was done by Pashov Audit Group, during which **0x37**, **BenRai**, **Drynooo**, **ihitshamsudo** engaged to review **Paxos Labs**. A total of 2 issues were uncovered.

Protocol Summary

Project Name	Paxos Labs
Protocol Type	Vault Management
Timeline	June 25th 2026 - June 27th 2026

Review commit hash:

- [f0007998e7e13ce34363f14b587a1a4d1d9f7aa8](#)
(paxoslabs/nucleus-boring-vault)

Fixes review commit hash:

- [094289219a4ace472e7b6be06187266c4aecaed3](#)
(paxoslabs/nucleus-boring-vault)

Scope

[TransitStation.sol](#) [EquivalentExchange.sol](#) [GenericDecoderAndSanitizer.sol](#)
[KhalaniDecoderAndSanitizer.sol](#) [NucleusDecoderAndSanitizer.sol](#)
[UniswapUniversalStablecoinDecoderAndSanitizer.sol](#) [Pausable.sol](#)
[AccessAuthority.sol](#) [Constants.sol](#) [DecoderCustomTypes.sol](#)



7. Findings

Findings count

Severity	Amount
Low	2
Total findings	2

Summary of findings

ID	Title	Severity	Status
[L-01]	Stale <code>executePendingOrders</code> signature prevents merkle verified execution	Low	Resolved
[L-02]	The protocol may charge zero fees for low-decimal stable coins	Low	Acknowledged



Low findings

[L-01] Stale `executePendingOrders` signature prevents merkle verified execution

Description

`NucleusDecoderAndSanitizer.executePendingOrders()` still declares the old signature `executePendingOrders(bytes32[] uuids, uint256[] amounts)`, but `TransitStation.executePendingOrders()` was reworked to take `FillBatch[] batches`, so the two hash to different 4-byte selectors. Because `ManagerWithMerkleVerification` keys each leaf on the target plus the call selector, any Merkle leaf built from this decoder points at a selector that no longer exists on the station — so a `BoringVault` routing `executePendingOrders()` through `manageVaultWithMerkleVerification()` (the path needed whenever the `wantAssetSource` vault is also the executor) fails verification and cannot fulfill orders. The decoder body is additionally empty and sanitizes no addresses, whereas each `FillBatch` now carries a `wantAsset` address that should be allowlisted by the root. Realign the decoder to `executePendingOrders(FillBatch[] batches)` so the selector matches the live function, and extract each `batch.wantAsset` into `addressesFound`.

[L-02] The protocol may charge zero fees for low-decimal stable coins

Description

According to the doc, only genuine 1:1 pegs can be routed. The Gemini USD is one standard ERC20 token, and also one genuine 1:1 peg with USD. This token's decimal is 2. Assume that the protocol fee is 0.1%; if users swap Gemini USD for 9 USD, the protocol fee will always be 0.

```
function _verifyAndCollect(
    Quote calldata quote,
    bytes calldata signature
)
    internal
    returns (bytes32 digest, uint256 offerAmountNormalized18AfterFees)
{
    uint256 maxProtocolFee = (quote.offerAmount * MAX_PROTOCOL_FEE_BPS) /
ONE_HUNDRED_PERCENT;
    if (quote.protocolFee > maxProtocolFee) revert ProtocolFeeTooHigh(quote.protocolFee,
maxProtocolFee);
}
```



Client Commentary

Acknowledged. We'll keep an eye out for these, but we figure no smart contract changes can be made here to easily solve this issue. We could introduce fee rounding to 1 wei in various ways or take fees optionally in want asset, but the complexity is not worth the tradeoff here. We'll instead carefully consider minimums.